

Lecture 7: Reinforcement Learning

Jelle Zuidema
ILLC, Universiteit van Amsterdam
(based on a slideset by Iris Proff)

Machine Learning and Language Models, 2025

Plan

What is RL?

Q-learning

RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

Types of learning algorithms

- Supervised learning
Learning from labeled examples – neural networks
- Unsupervised learning
Findings patterns in the raw data - clustering
- Semi-supervised learning
Learning from reward signals – Reinforcement learning

Reinforcement learning

A structural model from behavioral psychology...

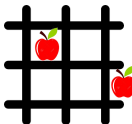
Pavlov: 1897 Pavlovian learning / classical conditioning
→ agent learns stimulus-outcome contingency



Thorndike: 1905 Law of effect / instrumental conditioning
→ agent learns action-outcome contingency



Tolman: 1930 Latent learning / goal directed learning
→ agent learns cognitive map



Reinforcement learning

...formalized by AI to a cognitive model

Structural model

Classical conditioning

Instrumental conditioning

Goal directed learning

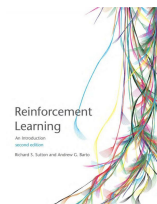
Cognitive model

→ Rescorla-Wagner model (1972)

→ Temporal difference learning (Samuel 1959)

→ Q-Learning (Watkins & Dayan 1992)

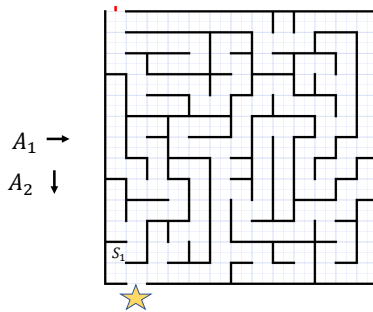
→ Model-based learning

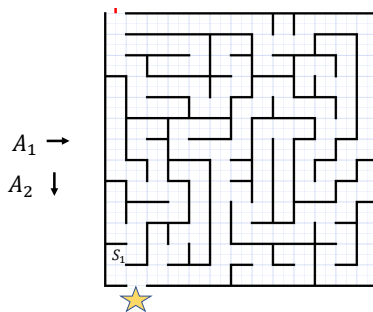


Determining utility (optimal control theory)



$$U(S) = R(S)$$

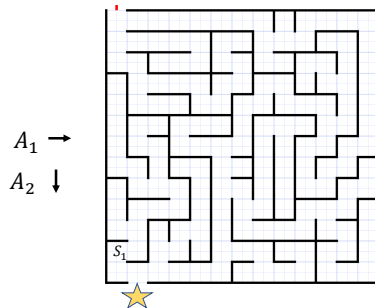




Question 1

Consider states S_2 and S_3 (which can you reach with action A_1 and A_2) and state S_4 (which you can reach from state S_3 by action A_1). How valuable are each of these states relative to each other? Is $U(S_1) = U(S_3)$?

Determining utility (optimal control theory)



Bellman equation:

$$U(S) = R(S) + \gamma * \max_A [U(S')]$$

U: utility

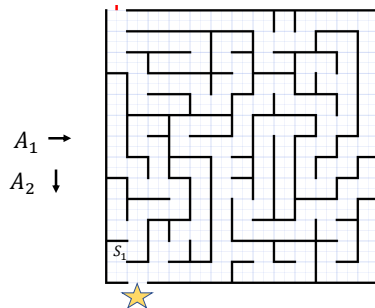
S: state

R: reward

A: action

γ : temporal discount factor

Determining utility (optimal control theory)



Bellman equation:

$$U(S) = R(S) + \gamma * \max_A [U(S')]$$

U: utility

S: state

R: reward

A: action

γ : temporal discount factor

Question 2

How do you avoid having to first learn everything about the environment/problem before you can start acting in the world?

Plan

What is RL?

Q-learning

RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

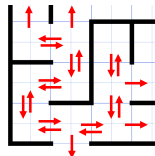
Q-Learning / temporal-difference learning

How to learn the utility of state-action pairs by trial-and-error?

Learning the Q-value = approximation of utility

$$Q(S, A) = \hat{U}(S, A)$$

Online learning (policy is applied and learned at the same time)



Q-Learning – update rule

Prediction error

$$\delta = R + \gamma * \max_{A'}(Q_t(S', A')) - Q_t(S, A)$$

Q-value update

$$Q_{t+1}(S, A) = Q_t(S, A) + \alpha * \delta$$

Q-value approximates utility

$$\lim_{t \rightarrow \infty} Q_t(S, A) = U(S, A)$$

Reminder: Bellman equation

$$U(S, A) = R(S, A) + \gamma * \max_{A'} [U(S', A')]$$

S: state

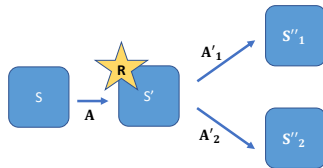
R: reward

A: action

γ : temporal discount factor

α : learning rate

U: utility



Q-learning – action selection policy

Say we are in state S , with possible actions $\mathbf{A}_1$ and $\mathbf{A}_2$, with
 $\mathbf{Q}(S, \mathbf{A}_1) = 0.5$ and $\mathbf{Q}(S, \mathbf{A}_2) = 0.7$

Which action should we choose?

Q-learning – action selection policy

Say we are in state S , with possible actions A_1 and A_2 , with
 $Q(S, A_1) = 0.5$ and $Q(S, A_2) = 0.7$

Which action should we choose?

Question 3

How do you make sure you maximize reward, both in the short term (taking advantage of your knowledge) and in the long term (extending your knowledge)?

Exploration – exploitation trade-off



exploit what you know

should you...

or

explore new options?



“A reinforcement learning agent has to exploit what it has already experienced in order to obtain reward, but it also has to explore in order to make better action selections in the future.

The dilemma is that neither exploration nor exploitation can be pursued exclusively without failing at the task.”

Sutton & Barto 2018

Exploration – exploitation trade-off



exploit what you know

should you...

or

explore new options?



“A reinforcement learning agent has to exploit what it has already experienced in order to obtain reward, but it also has to explore in order to make better action selections in the future.

The dilemma is that neither exploration nor exploitation can be pursued exclusively without failing at the task.”

Sutton & Barto 2018

Classic example: **multi-armed bandit**

Q-learning – action selection policy

Say we are in state S , with possible actions A_1 and A_2 , with
 $Q(S, A_1) = 0.5$ and $Q(S, A_2) = 0.7$

Which action should we choose, to trade off exploration and exploitation?

Epsilon greedy: $P(a_i|S) = \begin{cases} \frac{1-\epsilon}{|max_{q_l}(A)|} & \text{for } q_i = max(A) \\ \frac{\epsilon}{|A|-|max_{q_l}(A)|} & \text{else} \end{cases}$ best action chosen with $P = 1 - \epsilon$

Softmax: $P(a_i|S) = \frac{e^{\frac{q_i}{\tau}}}{\sum_A e^{\frac{q_i}{\tau}}}$ for $\tau \rightarrow 0$ greedy, for $\tau \rightarrow \infty$ random

Q-Learning – pseudo code

```
Set  $\alpha \in (0,1]$ , small  $\varepsilon > 0$ ,  $\gamma > 0$   
Initialize  $Q(s, a)$  for all  $s \in S, a \in A$  to 0  
Initialize  $s$   
For number of episodes  
  While  $s$  is not a terminal state  
    choose  $a$  from  $s$  using action selection policy  
    observe  $R$  and  $s'$   
    update  $Q$   
    update  $s$  to  $s'$ 
```

Plan

What is RL?

Q-learning

RL in the brain

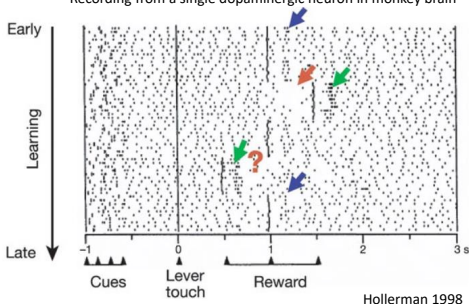
Model-based RL

Extensions of Q-learning

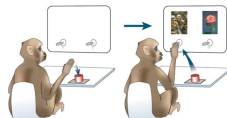
Deep RL

Temporal difference learning in the brain

Recording from a single dopaminergic neuron in monkey brain



Hollerman 1998



Upper blue: expected reward

Red: expected reward omitted

Green: unexpected early/late reward

Lower blue: expected reward

Plan

What is RL?

Q-learning

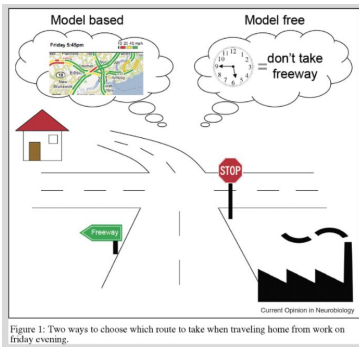
RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

Model-based vs model-free learning

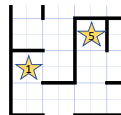
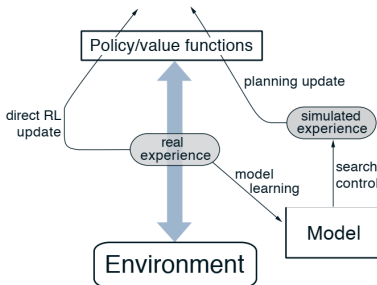


Model-free (habitual): rely on trial-and-error, efficient but inflexible

Model-based (goal directed): build a model of the environment, effortful but flexible

Dayan & Niv 2008

Model-based reinforcement learning



Model: $(s, a) \rightarrow (s', r)$

»What does my environment look like? «

Sutton & Barto 2018

Dyna-Q – pseudo code

initialize parameters

for number of episodes

while s is not a terminal state

choose a from s using action selection policy

observe R and s' , update Q , update s

update model $M(s,a) = (R,s')$

repeat n times

choose (s,a) to simulate

simulate action: observe R and s'

update Q

direct RL update

real experience

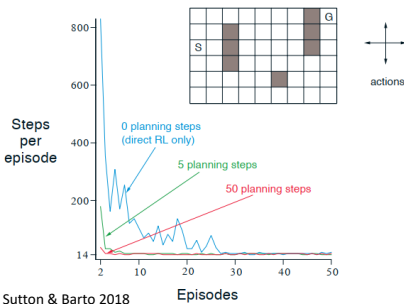
model learning

search control

planning update

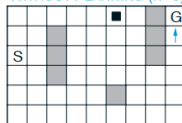
planning

Model-based learning *Dyna-Q*

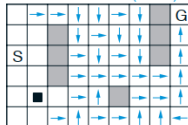


policy during episode 2

WITHOUT PLANNING ($n=0$)



WITH PLANNING ($n=50$)



Plan

What is RL?

Q-learning

RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

Extensions of RL algorithms

- Eligibility traces

update not only last state-action pairs, but all previously visited state-action pairs with decay factor λ

- Decaying learning rate

α decreases over time

- Hybrid models

Model-based and model-free component

...

Plan

What is RL?

Q-learning

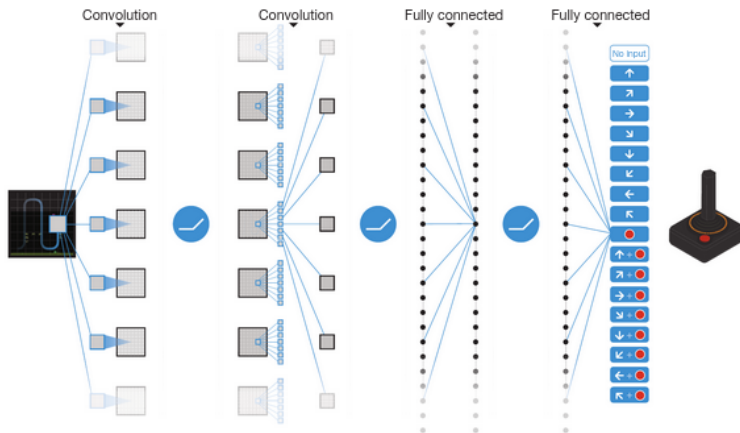
RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

Deep Reinforcement Learning



Deep Q-Learning (Mnih et al. 2015)

- Replace Q-table with a neural network Q , with weights θ ; use it to compute Q-value of current state
- Make a copy of the Q-network (the *target-network* $\hat{Q}$); use it to compute Q-value of target states
- Learn in batches: act in the world (choose actions using ϵ -greedy) and gather data
- Optimize Q-network (mimize prediction error) via gradient descent on a batch of observations
- Occasionally copy Q into $\hat{Q}$

Deep Q-Learning (Mnih et al. 2015)

Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$

For episode = 1, M **do**

Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ **do**

With probability ε select a random action a_t

otherwise select $a_t = \arg\max_a Q(\phi(s_t), a; \theta)$

Execute action a_t in emulator and observe reward r_t and image x_{t+1}

Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

Every C steps reset $\hat{Q} = Q$

End For

End For

Deep Q-Learning (Mnih et al. 2015)

LETTER

Human-level control through deep reinforcement learning

Volodymyr Mnih¹, Koray Kavukcuoglu², David Silver¹, Andrei A. Rusu¹, Joel Veness¹, Marc G. Bellemare¹, Alex Graves¹, Martin Riedmiller¹, Andreas K. Fiedler¹, Georg Ostrovski¹, Stig Petersen¹, Charles Beattie¹, Amir Sadik¹, Ioannis Antonoglou¹, Helen King¹, Dhruv Bansal¹, Dan Wierstra¹, Shane Legg¹, Demis Hassabis¹

doi:10.1038/nature14246

The theory of reinforcement learning provides a normative account, deeply rooted in psychological and neuroscientific perspectives on animal behaviour, of how agents may optimize their control of an environment. To use reinforcement learning successfully in situations approaching real-world complexity, however, agents are confronted with a difficult task: they must develop efficient representations of the environment from high-dimensional sensory inputs, and use these to generalize past experience to new situations. Remarkably, humans and other animals seem to solve this problem through a harmonious combination of reinforcement learning and hierarchical sensory processing systems¹, the former evidenced by a wealth of neural data revealing suitable parallels between the phasic signals emitted by dopaminergic neurons and temporal difference reinforcement learning algorithms². While reinforcement learning agents have achieved some successes in a variety of domains³, their applicability has previously been limited to domains in which useful features can be handcrafted, or in domains with fully observed, low-dimensional state spaces. Here we use recent advances in training deep neural networks^{4,5} to develop a novel artificial agent, termed a deep Q-network, that can learn optimal policies directly from high-dimensional sensory inputs and to end-to-end reinforcement learning. We tested this agent on the challenging domain of classic Atari 2600 games⁶. We demonstrate that the deep Q-network agent, receiving only the pixels and the game score as inputs, was able to surpass the performance of all previous algorithms and achieve a level comparable to that of a professional human player in scores of 49 games, using the same algorithm, network architecture and hyperparameters. This work bridges the divide between high-dimensional sensory inputs and actions, resulting in the first artificial agent that is capable of learning to excel at a diverse array of challenging tasks.

We set out to create a single algorithm that is able to develop a wide range of competencies on a varied range of challenging tasks—a central goal of general artificial intelligence⁷ that has eluded previous attempts^{8–10}. To achieve this, we developed a novel agent, a deep Q-network (DQN), which is able to combine reinforcement learning with a class of artificial neural networks¹¹ known as deep convolutional networks. Notable advances in deep neural networks^{12–14}, in which several layers of nodes are used to build up progressively more abstract representations of the data, have made it possible for artificial neural networks to learn concepts such as object categories directly from raw sensory data. We use particularly successful architectures, the deep convolutional network¹⁵, which uses hierarchical layers of local convolutional filters to extract the features of receptive fields—inspired by Hubel and Wiesel’s seminal work on hierarchical processing in early visual cortex¹⁶. Relying on the local spatial correlations present in images, and building in robustness to natural variations in the location of important or scarce.

We consider tasks in which the agent interacts with an environment through a sequence of observations, actions and rewards. The goal of the

agent is to select actions in a fashion that maximizes cumulative future reward. More formally, we use a deep convolutional neural network to approximate the optimal action-value function

$$Q^*(s,a) = \max_b \mathbb{E} [r + \gamma V^*(s') + \gamma^2 r' + \dots] = \max_b \sum_{t=0}^{\infty} \gamma^t r_{t+1} | s, a$$

in which the maximum sum of rewards, discounted by γ at each time step t , achievable by a behaviour policy $\pi = P(a_t|s_t)$, after making an observation (s) and taking an action (a) (see Methods¹⁷). Reinforcement learning is known to be unstable or even to diverge when a nonlinear function approximator such as a neural network is used to represent the action-value (also known as Q function). This instability has several causes: the correlations present in the sequence of observations, the fact that small updates to Q may significantly change the policy and therefore change the data distribution, and the correlations between the action values ($Q(s)$) and the target values $r + \gamma \max_b Q(s')$. We address these instabilities with a novel variant of Q learning, which uses two key ideas. First, we used a biologically inspired mechanism termed experience replay¹⁸ that randomizes over the data, thereby removing correlations in the observation sequence and smoothing out changes in the data distribution (see below for details). Second, we used an iterative update that adjusts the action-values (Q) towards target values that are only periodically updated, thereby reducing correlations with the target.

While other viable methods exist for training neural networks in the reinforcement learning setting, such as neural fitted Q -iteration¹⁹, these methods require the repeated training of networks across thousands of iterations. Consequently, these methods, unlike our algorithm, are too inefficient to be used successfully with large neural networks. We parameterize the approximate value function ($Q(s,a)$) using the same deep convolutional neural network shown in Fig. 1, in which f is the parameter in which, in the discount factor determining the agent’s behavior, f are the parameters of the Q -network at iteration t , f' are the network parameters used to compute the target at iteration t . The target network parameters f' are only updated with the Q -network parameters (b) every C steps and are held fixed between individual updates (see Methods).

To evaluate our DQN agent, we took advantage of the Atari 2600 platform, which offers a diverse array of tasks ($n = 49$) designed to be

$$L(f) = \mathbb{E}_{(s,a,r,s')} \left[\left(r + \gamma \max_{b'} Q(f',s') - Q(f,s,a) \right)^2 \right]$$

NEWS & VIEWS

Learning to see and act

An artificial intelligence system uses machine learning from massive training sets to teach itself to play 49 classic computer games, demonstrating that it can adapt to a variety of tasks. See Letters p. 527

BERNHARD SCHÖLKOPF

Improvements in our ability to process large amounts of data have led to progress in many areas of science, not least artificial intelligence (AI). With advances in machine learning has come the development of machines that can learn intelligent behaviour directly from data, rather than being explicitly programmed to exhibit such behaviour. For instance, the advent of ‘big data’ has resulted in systems that can recognize objects and words with considerable precision. On page 520 of this issue, Mnih et al. describe an agent that uses large data sets to teach itself how to play 49 classic Atari 2600 computer games by looking at the pixels and learning actions that increase the game score. It beat a professional games player in many instances – a remarkable example of the progress being made in AI.

In machine learning, systems are trained to infer patterns from observational data. A particularly simple type of pattern, a mapping between input and output, can be learnt through a process called supervised learning. A supervised learning system is given training data consisting of example inputs and the corresponding outputs, and comes up with a

model to explain these data. It does this by choosing from a class of model specified by the system designer. Designing this class is an art: its size and complexity should reflect the amount of training data available, and its content should reflect ‘prior knowledge’ that the designer of the system considers useful for the problem at hand. If all this is done well, the inferred model will then apply not only for the training set, but also for new data that adhere to the same underlying pattern.

The rapid growth of data sets means that machine learning can now use complex model classes and handle highly non-trivial inference problems. Such problems are usually characterized by several factors: the data are multidimensional, the underlying pattern is complex (or chaotic), it might be nonlinear at the input, and the designer has only weak prior knowledge about the problem – in particular, a mechanistic understanding is lacking. The human brain repeatedly solves non-trivial inference problems as we about our daily lives, using high-dimensional sensory data to determine how best to control all the muscles of the body. Single supervised learning is clearly not the whole story, because

we often learn without a ‘supervisor’ telling us the output for a hypothetical input-output function. Here, ‘reinforcement’ has a central role in learning behaviour from weaker supervisors. Machine learning adopted this idea to develop reinforcement learning algorithms, in which supervisors take the form of a numerical reward signal² – and the goal is for the system to learn a policy that, given the current state, determines which action to pick to maximize an accumulated future reward.

Mnih et al. use a form of reinforcement learning known as Q -learning³ to teach systems to play a set of 49 vintage video games, learning how to increase the game score as a numerical reward. In Q -learning, Q is an estimate of the accumulated future reward, Q^* if it states the system first performs action a , and subsequently follows an optimal policy. The system tries to approximate Q^* by using an artificial neural network – a biological analogue (see) inspired by a function approximator known as the deep Q -network (DQN). The DQN’s input (the pixels from four consecutive game screens) is processed by connected ‘hidden’ layers of computations, which extract more and more specialized visual features to help approximate the complex

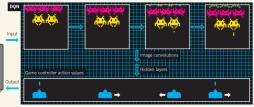


Figure 1 | Computer games. Mnih et al. have designed an artificial intelligence system, using a deep Q-network (DQN), that learns how to play 49 video games. The DQN analyses a sequence of game screen captures (consecutive and approximated) and then generates the action taken as the consequence on the future game screen of that action is taken and followed by the best possible course of subsequent actions. The first layers of the DQN

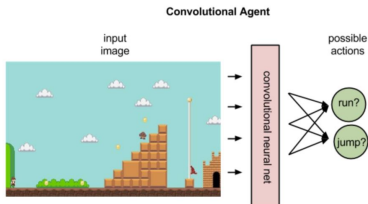
analyse the pixels of the game screen and extract information from more and more specialized visual features (image correlations). Subsequently, fully connected hidden layers predict the value of actions from these features. The last layer is the output layer – the action taken by the DQN. The possible outputs depend on the specific game the system is playing; everything else is the same in each of the 49 games.

¹Google DeepMind, 11 Street View House Number, London EC2A 4PB, UK.

²These authors contributed equally to this work.

Deep Reinforcement Learning

Learn Q-values with deep neural nets



[A nice tutorial on deep reinforcement learning](https://wiki.pathmind.com/deep-reinforcement-learning)

<https://wiki.pathmind.com/deep-reinforcement-learning>